

DOCUMENTO DE

DISEÑO Y ESPECIFICACIONES

Juego de Ajedrez Online

Daniel Reyes, Oscar Peñuela, Miguel Fernández, Andres Baquero

Fecha: 17/3/2026

Versión: 1.1

1. Introducción.....	3
2. Interfaz (Mockup).....	3
2.1 Menú inicial.....	3
2.2 Pantalla de juego.....	3
3. Módulo 1: Motor de Ajedrez (Game Engine).....	4
3.1 Movimiento de Piezas.....	4
3.2 Reglas Especiales.....	5
3.3 Detección de Estados del Juego.....	5
3.4 Generación de Movimientos Legales.....	6
4. Módulo 2: Autenticación y Usuarios.....	7
4.1 Registro.....	7
4.2 Inicio de Sesión.....	7
4.3 Finalizar Sesión.....	7
4.4 Perfil Público.....	7
4.5 Modo Invitado.....	8
5. Módulo 3: Matchmaking.....	9
6. Módulo 4: Gestión de Partidas (Game Session).....	9
6.1 Asignación Partida.....	9
6.2 Ciclo de Vida de la Partida.....	9
6.3 Finalizar en Partida.....	10
6.4 Desconexión y Reconexión.....	10
7. Módulo 5: Comunicación en Tiempo Real.....	11
8. Módulo 6: Historial de Partidas y Rating ELO.....	11
8.1 Historial.....	11
8.2 Sistema ELO.....	12
9. Módulo 7: IA para Partidas Offline.....	12
10. Módulo 8: Social - Amigos y Chat.....	12
11. Descripción de Servicios.....	14
12. Mapa de Comunicación entre Servicios.....	15
13. Diagrama Entidad - Relación.....	17
14. Fases de Desarrollo.....	17
Fase 1: Aplicación web base y arquitectura del sistema.....	17
Fase 2: Consolidación funcional del juego.....	17
Fase 3: Funcionalidades avanzadas y optimización.....	17

1. Introducción

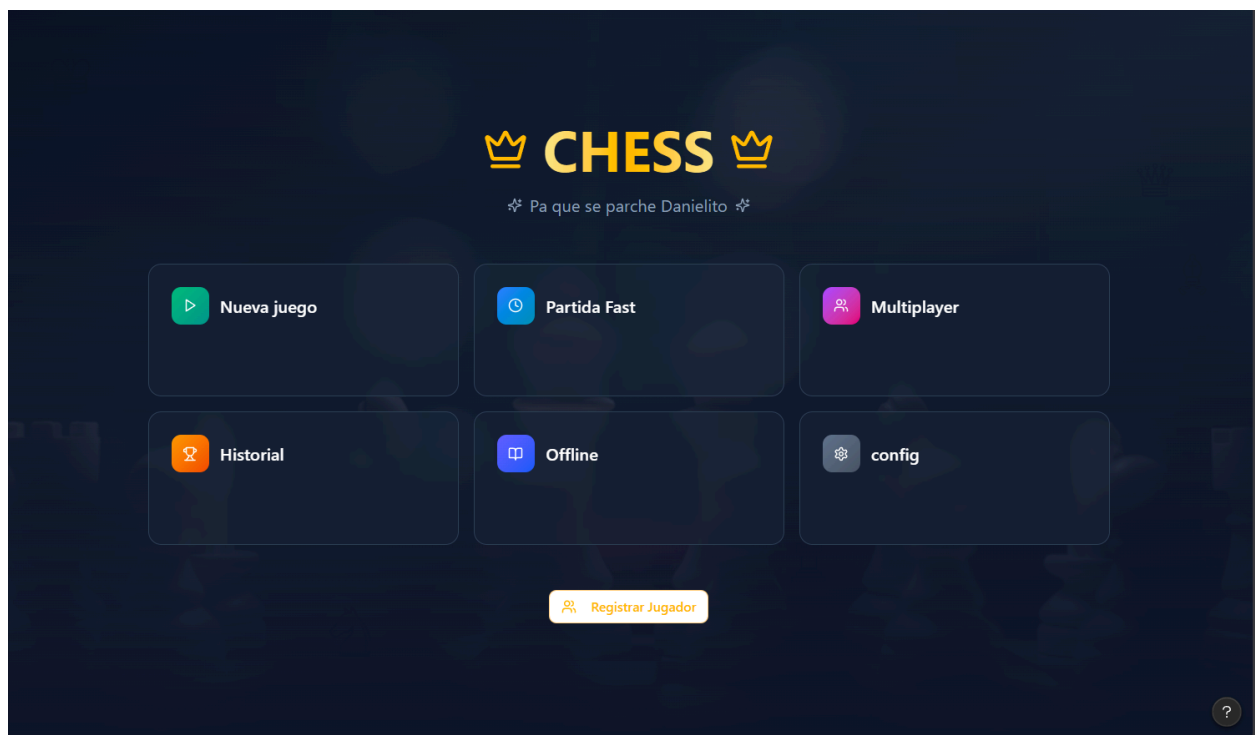
Este documento presenta de forma unificada la propuesta de desarrollo de un juego de ajedrez online. Reúne los requerimientos funcionales, la arquitectura del sistema, la descripción de los servicios, el diagrama entidad-relación y las fases de implementación, con el fin de ofrecer una visión clara y ordenada del proyecto.

2. Interfaz (Mockup)

Estas vistas permiten ilustrar la estructura general de la interfaz, la disposición de los elementos principales y el flujo básico del juego, aunque no corresponden necesariamente a la versión final.

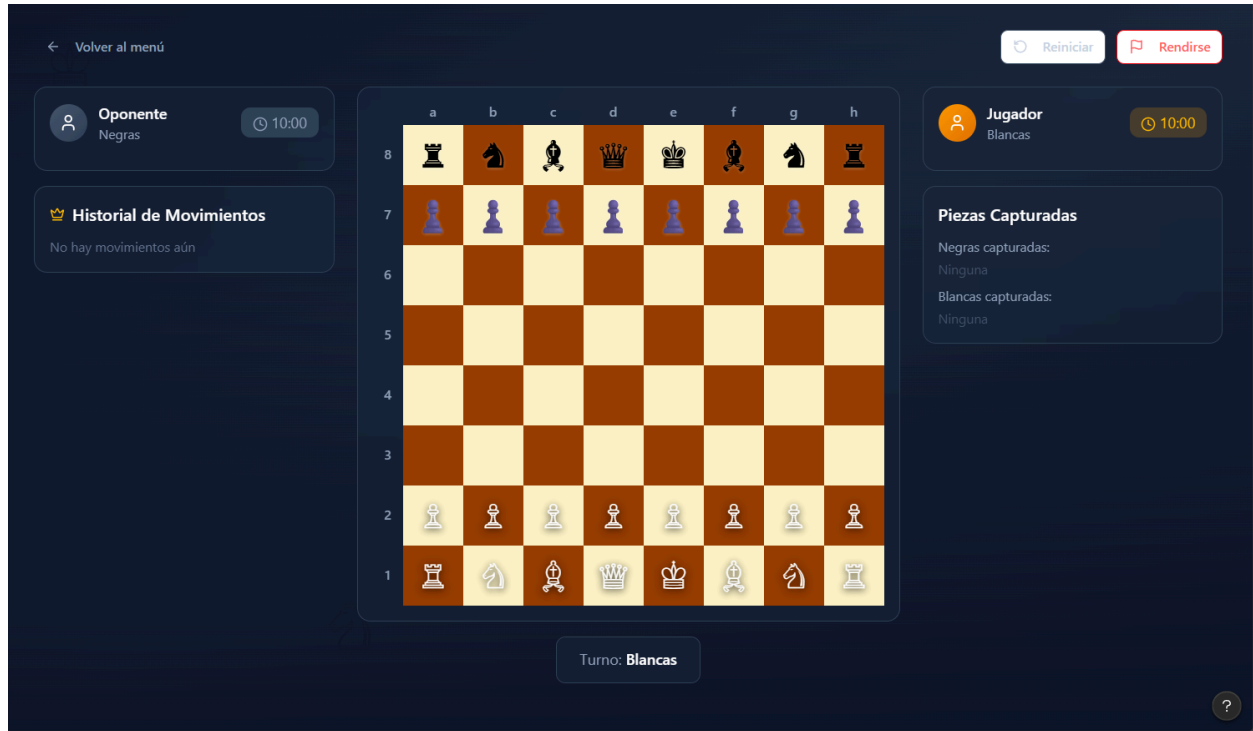
2.1 Menú inicial

Aquí se observan las opciones iniciales disponibles para el usuario, así como el punto de entrada a las distintas funcionalidades del juego.



2.2 Pantalla de juego

Esta vista muestra la interfaz general del tablero y los elementos principales presentes durante una partida.



Para una mejor visualización, se puede visitar el siguiente enlace:
<https://www.figma.com/make/cmvsz97Nl49PNyxs6Z4ZS/Chess-Menu?p=f>

3. Módulo 1: Motor de Ajedrez (Game Engine)

Este es el núcleo del sistema. Toda la lógica de reglas del ajedrez vive aquí. Ningún otro módulo valida movimientos; todos delegan al motor.

3.1 Movimiento de Piezas

ID	Descripción	Entrada	Salida	Prioridad
RF-ENG-001	Validar movimiento del peón: avance 1 casilla, avance 2 desde posición inicial, captura en diagonal.	Pieza, casilla origen (fila, col), casilla destino (fila, col), estado del tablero	Movimiento válido/inválido + tablero actualizado	Alta
RF-ENG-002	Validar movimiento de la torre: líneas rectas horizontal/vertical sin obstáculos.	Pieza, origen, destino, estado del tablero	Movimiento válido/inválido + tablero actualizado	Alta
RF-ENG-003	Validar movimiento del alfil: diagonales sin obstáculos.	Pieza, origen, destino, estado del tablero	Movimiento válido/inválido + tablero actualizado	Alta

RF-ENG-004	Validar movimiento de la reina: combina torre + alfil.	Pieza, origen, destino, estado del tablero	Movimiento válido/inválido + tablero actualizado	Alta
RF-ENG-005	Validar movimiento del caballo: forma de L, puede saltar piezas.	Pieza, origen, destino, estado del tablero	Movimiento válido/inválido + tablero actualizado	Alta
RF-ENG-006	Validar movimiento del rey: 1 casilla en cualquier dirección, no puede moverse a casilla amenazada.	Pieza, origen, destino, estado del tablero	Movimiento válido/inválido + tablero actualizado	Alta

3.2 Reglas Especiales

ID	Descripción	Entrada	Salida	Prioridad
RF-ENG-007	Enroque corto (O-O): Rey y torre del lado del rey. Condiciones: ninguna pieza ha movido, no hay piezas entre ellas, el rey no está en jaque, no pasa por casilla amenazada.	Rey seleccionado, intento de enroque, historial de movimientos	Enroque ejecutado o rechazado con razón	Alta
RF-ENG-008	Enroque largo (O-O-O): Igual que enroque corto pero con torre del lado de la reina.	Rey seleccionado, intento de enroque, historial de movimientos	Enroque ejecutado o rechazado con razón	Alta
RF-ENG-009	Captura al paso (en passant): Un peón puede capturar a otro peón que acaba de avanzar 2 casillas y quedó a su lado.	Peón, destino, último movimiento del oponente	Captura ejecutada o rechazada	Alta
RF-ENG-010	Promoción de peón: Al llegar a la última fila, el jugador elige pieza (reina, torre, alfil o caballo).	Peón en última fila, pieza elegida por el jugador	Peón reemplazado por pieza seleccionada	Alta

3.3 Detección de Estados del Juego

ID	Descripción	Entrada	Salida	Prioridad
RF-ENG-011	Detectar jaque: El rey de un jugador está amenazado. El jugador debe resolver el jaque en su siguiente turno.	Estado del tablero después de cada movimiento	Booleano: jaque sí/no + piezas que amenazan	Alta
RF-ENG-012	Detectar jaque mate: El rey está en jaque y no hay movimiento legal que lo resuelva. Fin de partida.	Estado del tablero, lista de movimientos legales del jugador en turno	Booleano: jaque mate sí/no + jugador ganador	Alta

RF-ENG-013	Detectar ahogado (stalemate): El jugador en turno no tiene movimientos legales pero NO está en jaque. Tablas.	Estado del tablero, movimientos legales	Booleano: ahogado sí/no	Alta
RF-ENG-014	Detectar tablas por repetición triple: La misma posición del tablero se repite 3 veces.	Historial completo de posiciones	Booleano: repetición sí/no	Media
RF-ENG-015	Detectar tablas por regla de 50 movimientos: 50 movimientos consecutivos sin captura ni avance de peón.	Contador de movimientos sin captura/peón	Booleano: regla 50 sí/no	Media
RF-ENG-016	Detectar tablas por material insuficiente: No hay piezas suficientes para dar jaque mate (ej: rey vs rey).	Piezas restantes en el tablero	Booleano: material insuficiente sí/no	Media

3.4 Generación de Movimientos Legales

ID	Descripción	Entrada	Salida	Prioridad
RF-ENG-017	Generar lista de movimientos legales para una pieza seleccionada, considerando jaques, clavadas y bloqueos.	Pieza seleccionada, estado del tablero	Array de coordenadas válidas [{fila, col}]	Alta
RF-ENG-018	Registrar cada movimiento en notación algebraica estándar (ej: Nf3, O-O, exd5).	Movimiento ejecutado	String en notación algebraica	Media

4. Módulo 2: Autenticación y Usuarios

Gestiona el registro, inicio de sesión y perfil público de los jugadores. Es prerequisite para partidas online y ranking.

4.1 Registro

ID	Descripción	Entrada	Salida	Prioridad
RF-AUTH-001	Registrar usuario con correo electrónico y contraseña. Validar formato de correo y fortaleza de contraseña (mín. 8 caracteres, 1 mayúscula, 1 número).	Correo, sobrenombre	ID de usuario, token de sesión, confirmación	Alta

4.2 Inicio de Sesión

ID	Descripción	Entrada	Salida	Prioridad
RF-AUTH-002	Iniciar sesión con credenciales válidas. Generar token JWT con expiración.	Correo, contraseña	Token JWT, datos básicos del usuario	Alta

4.3 Finalizar Sesión

ID	Descripción	Entrada	Salida	Prioridad
RF-AUTH-003	Cerrar sesión e invalidar token activo.	Token de sesión	Confirmación de cierre	Alta

4.4 Perfil Público

ID	Descripción	Entrada	Salida	Prioridad
RF-AUTH-004	Mostrar perfil público del jugador: sobrenombre, rating ELO, partidas jugadas, porcentaje de victorias.	ID del jugador	Objeto con datos públicos del perfil	Media
RF-AUTH-005	Permitir editar sobrenombre y avatar (opcional).	ID del jugador, nuevos datos	Perfil actualizado	Baja

4.5 Modo Invitado

ID	Descripción	Entrada	Salida	Prioridad
RF-AUTH-006	Permitir acceso sin registro para partidas locales (mismo dispositivo, 2 jugadores) y partidas contra IA.	Ninguna (acceso directo)	Sesión temporal sin persistencia	Alta
RF-AUTH-007	El invitado NO puede acceder a: matchmaking online, historial de partidas, ranking ELO.	Sesión de invitado	Acceso restringido con mensaje informativo	Alta

5. Módulo 3: Matchmaking

Empareja jugadores de nivel similar para partidas en línea. Requiere usuario autenticado.

ID	Descripción	Entrada	Salida	Prioridad
RF-MM-001	Agregar jugador a la cola de búsqueda con su rating ELO y modo de juego seleccionado (rápida, clásica).	ID del jugador, rating ELO, modo de juego	Confirmación de entrada a la cola	Alta
RF-MM-002	Emparejar jugadores con diferencia de ELO ≤ 200 puntos. Si no hay coincidencia en 30 segundos, ampliar rango a ≤ 400 .	Cola de jugadores en espera	Par de jugadores emparejados + ID de partida	Alta
RF-MM-003	Permitir cancelar búsqueda de oponente antes de ser emparejado.	ID del jugador en cola	Jugador removido de la cola	Alta

6. Módulo 4: Gestión de Partidas (Game Session)

Administra el ciclo de vida completo de una partida: creación, turnos, temporizador, desconexiones y finalización.

6.1 Asignación Partida

ID	Descripción	Entrada	Salida	Prioridad
RF-GAME-001	Crear partidas que tengan reconocimiento específico, con un id propio.	IDs de ambos jugadores, modo de juego	ID de partida.	Alta
RF-GAME-002	Asignar cada jugador en el lado que va a jugar en la partida.	ID's jugador, ID partida	Asignación color	Media

6.2 Ciclo de Vida de la Partida

ID	Descripción	Entrada	Salida	Prioridad
RF-GAME-003	Crear partida con estado inicial: tablero en posición inicial, turno de blancas, temporizadores configurados según modo de juego.	IDs de ambos jugadores, modo de juego	ID de partida, estado inicial completo	Alta
RF-GAME-004	Procesar movimiento: recibir movimiento, delegarlo al Motor de Ajedrez para validación, actualizar estado y cambiar turno.	ID de partida, jugador, movimiento (origen, destino)	Estado actualizado: tablero, turno, tiempo restante, notación	Alta

RF-GAME-005	Gestionar temporizador por jugador. El reloj del jugador activo disminuye. Si llega a 0, pierde la partida.	ID de partida, turno activo	Tiempo restante actualizado o fin de partida por tiempo	Alta
RF-GAME-006	Finalizar partida por: jaque mate, tiempo agotado, rendición, tablas por acuerdo, tablas automáticas (ahogado, repetición, 50 mov, material).	Evento de finalización	Resultado (victoria blancas/negras/tablas), razón, estado final	Alta

6.3 Finalizar en Partida

ID	Descripción	Entrada	Salida	Prioridad
RF-GAME-007	Permitir al jugador rendirse (resign) en cualquier momento de su turno.	ID de partida, ID del jugador	Partida finalizada con victoria del oponente	Alta
RF-GAME-008	Permitir ofrecer tablas al oponente. El oponente puede aceptar o rechazar.	ID de partida, ID del jugador que ofrece	Oferta enviada al oponente / respuesta	Media
RF-GAME-009	Mostrar historial de movimientos de la partida en curso en notación algebraica.	Estado actual de la partida	Lista ordenada de movimientos	Media

6.4 Desconexión y Reconexión

ID	Descripción	Entrada	Salida	Prioridad
RF-GAME-010	Si un jugador se desconecta, iniciar temporizador de gracia de 60 segundos. Si no reconecta, pierde.	Evento de desconexión, ID de partida	Temporizador de gracia activo / partida finalizada	Alta
RF-GAME-011	Permitir reconectar a una partida en curso. Restaurar estado completo del tablero y tiempos.	ID del jugador, ID de partida	Estado completo restaurado	Alta

7. Módulo 5: Comunicación en Tiempo Real

Define cómo se transmiten los datos entre cliente y servidor durante una partida en vivo. Se recomienda WebSockets para conexión bidireccional persistente.

ID	Descripción	Entrada	Salida	Prioridad
RF-RT-001	Establecer conexión WebSocket al iniciar partida. Mantener canal abierto hasta finalización.	ID de partida, token del jugador	Conexión establecida o error de autenticación	Alta
RF-RT-002	Transmitir movimientos en tiempo real al oponente con latencia < 200ms.	Movimiento validado por el motor	Evento recibido por el cliente del oponente	Alta
RF-RT-003	Sincronizar relojes de ambos jugadores cada 5 segundos para evitar desfases.	Tiempo del servidor	Tiempos sincronizados en ambos clientes	Alta
RF-RT-004	Enviar notificaciones en tiempo real: oferta de tablas, rendición, desconexión del oponente.	Evento de la partida	Notificación al cliente correspondiente	Alta

8. Módulo 6: Historial de Partidas y Rating ELO

Persiste resultados de partidas y calcula el rating de cada jugador.

8.1 Historial

ID	Descripción	Entrada	Salida	Prioridad
RF-HIST-001	Guardar datos de cada partida finalizada: jugadores, resultado, movimientos, duración, modo de juego, fecha.	Datos de partida finalizada	Registro persistido con ID único	Alta
RF-HIST-002	Listar partidas anteriores de un jugador con filtros: ganadas, perdidas, tablas, por fecha, por modo.	ID del jugador, filtros	Lista paginada de partidas	Media
RF-HIST-003	Permitir revisar una partida pasada movimiento por movimiento (replay).	ID de la partida	Secuencia de estados del tablero	Media

8.2 Sistema ELO

ID	Descripción	Entrada	Salida	Prioridad
RF-ELO-001	Calcular nuevo rating ELO tras cada partida usando fórmula estándar ($K=32$ para < 30 partidas, $K=16$ para ≥ 30).	Rating actual de ambos jugadores, resultado	Nuevo rating de ambos jugadores	Alta
RF-ELO-002	Asignar rating inicial de 1200 a todo jugador nuevo.	Registro de nuevo jugador	Rating = 1200	Alta
RF-ELO-003	Mostrar ranking global de jugadores ordenado por ELO, con posición del jugador actual resaltada.	Solicitud de ranking	Lista paginada de jugadores con rating	Baja

9. Módulo 7: IA para Partidas Offline

Proporciona un oponente artificial para la práctica. No requiere conexión a internet una vez cargado el cliente.

ID	Descripción	Entrada	Salida	Prioridad
RF-IA-001	Permitir al jugador elegir nivel de dificultad: Fácil, Medio, Difícil.	Selección del jugador	IA configurada al nivel elegido	Media
RF-IA-002	La IA genera movimientos legales dentro de un tiempo máximo de 3 segundos por turno.	Estado del tablero, nivel de dificultad	Movimiento seleccionado por la IA	Media
RF-IA-003	Nivel Fácil: Selecciona movimientos legales aleatorios con preferencia por capturas. Nivel Medio: Evaluación de material + posición (profundidad 2-3). Nivel Difícil: Algoritmo Minimax con poda alfa-beta (profundidad 4-5).	Estado del tablero, nivel	Movimiento óptimo según nivel	Media

10. Módulo 8: Social - Amigos y Chat

10.1 Amigos

ID	Descripción	Entrada	Salida	Prioridad
----	-------------	---------	--------	-----------

RF-AMI-001	El sistema debe permitir que un usuario envíe una solicitud de amistad a otro usuario registrado.	ID del usuario que envía la solicitud, ID del usuario que la recibe.	Confirmación de solicitud enviada.	Media
RF-AMI-002	El sistema debe permitir que un usuario acepte una solicitud de amistad recibida.	ID del usuario, ID de la solicitud de amistad.	Confirmación de amistad registrada.	Media
RF-AMI-003	El sistema debe permitir que un usuario rechace una solicitud de amistad recibida.	ID del usuario, ID de la solicitud de amistad.	Confirmación de solicitud rechazada.	Baja
RF-AMI-004	El sistema debe permitir consultar la lista de amigos de un usuario.	ID del usuario.	Lista de amigos registrados.	Media

10.2 Chat

ID	Descripción	Entrada	Salida	Prioridad
RF-CHAT-001	El sistema debe permitir enviar mensajes de chat entre jugadores dentro de una partida activa.	ID de la partida, ID del usuario emisor, contenido del mensaje.	Confirmación de mensaje enviado.	Alta
RF-CHAT-002	El sistema debe permitir mostrar en tiempo real los mensajes enviados dentro de una partida.	ID de la partida.	Mensajes recibidos en el chat de la partida.	Alta
RF-CHAT-003	El sistema debe permitir consultar el historial reciente de mensajes asociados a una partida.	ID de la partida.	Lista de mensajes enviados en la partida.	Media
RF-CHAT-004	El sistema debe permitir consultar la lista de amigos de un usuario.	Mensaje registrado en el sistema.	Nombre del remitente, contenido del mensaje y fecha/hora de envío.	Media

11. Descripción de Servicios

Servicio	Qué hace	Precondición	Qué recibe	Qué entrega
Servicio de Usuarios	Gestiona la creación de usuarios, y la consulta de estos en el sistema.	No existe una cuenta activa con el mismo correo o nombre de usuario	correoElectronico: String nombreUsuario: String	usuarioId: UUID estadoUsuario: String
Servicio de Autenticación	Gestiona el inicio de sesión, la validación de credenciales y la emisión de tokens.	El usuario tiene credenciales registradas y la cuenta está habilitada.	correoElectronico: String contrasena: String	sesionAutenticada: JSON Ejemplo: { usuarioId, tokenJwt, expiracionToken }
Servicio de Perfil	Mantiene y consulta la información pública o personalizable del jugador.	Usuario autenticado y perfil existente.	usuarioId: UUID	perfilPublico: JSON Ejemplo: { aliasUsuario, avatarUrl, descripcion, puntuacionELO }
Servicio de Matchmaking	Busca y asigna un oponente compatible según la modalidad de juego y criterios de emparejamiento.	Usuario autenticado, disponible y sin partida activa.	solicitudEmparejamiento: JSON Ejemplo: { usuarioId, modalidadPartida, controlTiempoSegundos }	emparejamiento: JSON Ejemplo: { partidaId, oponenteId, colorAsignado, estadoEmparejamiento }
Servicio de Motor de Ajedrez	Valida movimientos y calcula el nuevo estado del tablero conforme a las reglas del juego.	Existe una partida activa y se recibe un movimiento sobre un estado válido del tablero.	estadoTablero: JSON movimiento: JSON Ejemplo: { casillaOrigen, casillaDestino, piezaPromocion }	resultadoMovimiento: JSON Ejemplo: { movimientoValido, estadoTableroActualizado, turnoSiguiente, estadoPartida, eventoEspecial }
Servicio de Modo Invitado	Permite crear una sesión temporal para jugar sin cuenta registrada.	No requiere autenticación previa.	aliasTemporal: String (opcional)	sesionInvitado: JSON Ejemplo: { invitadoId, tokenInvitado, estadoSesion }
Servicio de Partidas	Gestiona el ciclo de vida de una partida y coordina la comunicación en tiempo real entre jugadores.	Emparejamiento confirmado o solicitud válida de inicio de partida.	eventoPartida: JSON Ejemplo: { partidaId, usuarioId, tipoAccion, payloadAccion }	estadoPartidaActualizado: JSON Ejemplo: { partidaId, estadoPartida, turnoActual, tiempoRestantePorJugador, resultadoFinal }

Servicio Historial	de	Almacena y consulta el registro de partidas finalizadas.	La partida terminó y su resultado fue validado.	partidaId: UUID	historialPartidaId: UUID registroGuardado: Boolean
Servicio Análisis Partidas	de	Evalúa una partida almacenada e identifica errores, mejores jugadas y patrones relevantes.	La partida ya se encuentra registrada en el historial.	partidaId: UUID	analisisPartida: JSON Ejemplo: { evaluacionGeneral, mejoresJugadas, erroresDetectados } }
Servicio Rating ELO	de	Calcula y actualiza el rating de los jugadores después de una partida finalizada.	El resultado final de la partida ya fue validado y registrado.	partidaId: UUID	actualizacionElo: JSON Ejemplo: [{ usuarioId, eloAnterior, eloNuevo, variacionElo }]
Servicio Amigos	de	Gestiona solicitudes de amistad, lista de amigos y relaciones sociales entre usuarios.	Usuario autenticado.	solicitudAmistad: JSON Ejemplo: { usuarioId, usuarioObjetivoId, tipoAccion } }	estadoRelacion: JSON Ejemplo: { estadoSolicitud, fechaActualizacion } }
Servicio de Chat		Gestiona mensajería entre usuarios o dentro de una partida/sala.	Usuario autenticado y sala de chat válido.	mensajeChat: JSON Ejemplo: { emisorId, salaId, contenido } }	mensajeEnviado: JSON Ejemplo: { mensajeId, fechaEnvio, estadoEntrega } }

12. Mapa de Comunicación entre Servicios

En esta sección se describe la interacción entre los servicios que conforman el sistema y la aplicación cliente, indicando los principales flujos de información, los datos intercambiados y el mecanismo de comunicación utilizado en cada caso. Su finalidad es ofrecer una visión general de la integración del sistema y evidenciar cómo se mantiene la separación de responsabilidades entre los distintos componentes.

Origen	Destino	Datos Transmitidos	Protocolo
Cliente (Frontend)	Servicio de Autenticación	Credenciales de acceso / registro	REST (HTTPS)
Servicio de Autenticación	Servicio de Usuarios	Datos base para creación/validación del usuario	REST (HTTPS)
Cliente (Frontend)	Servicio de Perfil	Solicitud de datos del perfil	REST (HTTPS)

Cliente (Frontend)	Servicio Matchmaking	de	Solicitud de emparejamiento / modalidad de juego	REST (HTTPS)	
Servicio Matchmaking	de	Servicio Autenticación	de	Validación de token y estado del usuario	REST (HTTPS)
Servicio Matchmaking	de	Servicio de Partidas	Datos de jugadores emparejados	REST / Evento interno	
Cliente (Frontend)	Servicio de Partidas	Conexión de partida / eventos en vivo	WebSocket		
Servicio de Partidas	Servicio de Motor de Ajedrez	Movimiento a validar + estado actual del tablero	API interna / invocación interna		
Servicio de Partidas	Servicio de Historial	Resultado de partida finalizada	REST / Evento interno		
Servicio de Historial	Servicio de Rating ELO	Datos de partida registrada para actualización de puntaje	REST / Evento interno		
Servicio de Historial	Servicio de Análisis de Partidas	Datos de partida para evaluación	REST / Evento interno		
Cliente (Frontend)	Servicio de Amigos	Solicitudes de amistad / consulta de amigos	REST (HTTPS)		
Cliente (Frontend)	Servicio de Chat	Mensajes de chat / consulta de historial reciente	WebSocket / REST		

13. Diagrama Entidad - Relación

El diagrama entidad relación se encuentra adjunto al final del documento.

14. Fases de Desarrollo

Se propone un desarrollo incremental en 3 fases:

Fase 1: Aplicación web base y arquitectura del sistema

En esta fase se prioriza la construcción de la aplicación web base y la separación modular de los servicios principales. Se implementarán de forma básica los módulos de usuarios, autenticación, perfil, amigos, partidas, matchmaking básico y modo invitado, de manera que el sistema permite registrarse, iniciar sesión, crear o unirse a partidas y visualizar el tablero desde la web. En esta etapa, el game engine será básico y no validará con total rigor las reglas del ajedrez.

Fase 2: Consolidación funcional del juego

En esta fase se incorporará la comunicación en tiempo real entre jugadores mediante WebSockets, junto con un temporizador simple y un registro mínimo de partidas. También se mejorará el game engine para ofrecer una validación más consistente de los movimientos. El objetivo será lograr una experiencia multijugador básica pero funcional dentro de la aplicación web. Igualmente, se hará una primera integración de IA como oponente en las partidas.

Fase 3: Funcionalidades avanzadas y optimización

En esta fase se fortalecerá la lógica del juego y se ampliarán los servicios complementarios del sistema. Se contempla mejorar el game engine, incorporar de forma más completa el historial de partidas y el sistema de rating ELO, y añadir funcionalidades avanzadas como análisis de partidas, estadísticas, rankings o IA con distintos niveles. Si el alcance lo permite, se integrarán opciones offline al juego.